KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 03

Accessing a Production Cluster & RBAC User Onboarding

Topic: Production Access + RBAC | CKA Weekend Course · Video 14, 7th Oct | 2026-09-11

The Real Question: How Do People Access K8s in Production?

While learning, we SSH straight into the `master` node and run `kubectl` there. **That is not how production works.** In production you almost never get direct SSH access to a manager/control-plane node. Instead:

- You get access to a **jump/bastion host** — a regular machine with a full OS that has network **connectivity** to the master.
- `kubectl` is installed on that jump host (not the app logic — just the CLI).
- You get a **kubeconfig file** *from* the cluster (generated by whoever administers it) and put it in your own `~/.kube/config` on the jump host.
- From then on, `kubectl` on the jump host talks to the cluster remotely — you never log into the master directly.

The 4 Steps to Get kubectl Access From Any New Host

1. Confirm **network connectivity** between the host and the cluster's API server.
2. **Install kubectl** on that host (e.g. `sudo snap install kubectl --classic`).
3. Get the **config file** from the cluster admin.
4. Create `~/.kube/` on your host and copy the config file in as `~/.kube/config`.

BASH

```
alok@gitserver:~$ #connectivity - ok
alok@gitserver:~$ #install kubectl - ok
alok@gitserver:~$ #get config file - ok
alok@gitserver:~$ #copy config ~/.kube
```

> 💡 **What's inside the config file**
>
> The cluster's certificate, the master server's IP/address, the cluster's name, and the username used to authenticate — that's the full picture of "who you are" and "where the cluster is."

Multi-Tenant Reality: Namespace-Scoped Access

A production cluster usually has multiple namespaces (e.g. `dev`, `test`, `prod`), and different users/teams should only access *their* namespace — not the whole cluster. This is where **RBAC** (Role-Based Access Control) comes in.

> 🏢 **Production note**
>
> Real production clusters usually plug into an existing directory service — **LDAP / Active Directory** — rather than creating individual Kubernetes users for every one of possibly thousands of employees. The cert-based flow below is what you'd use for a smaller/manual setup, or to understand the mechanics AD-integration builds on top of.

RBAC = a 2-Step Process

1. **Create a `Role` (namespace-scoped) or `ClusterRole` (cluster-wide).**
   - `Role` → e.g. "Alok can create pods in the `prod` namespace only."
   - `ClusterRole` → e.g. "Alok can create pods in *any* namespace."
2. **Attach it to the user** (a `RoleBinding` or `ClusterRoleBinding`).

That's the whole model — everything below is just the mechanics of steps 1 and 2, plus how a user identity gets created in the first place.

Creating a Certificate-Based User ("ballu") End to End

Step 1 — user generates a private key + CSR on their own workstation

```bash
openssl genrsa -out ballu.key 2048
openssl req -new -key ballu.key -out ballu.csr -subj "/CN=ballu"
```

`.csr` = Certificate Signing Request — "here's who I claim to be, please sign it."

Step 2 — user sends the .csr to the cluster admin (root CA)

```bash
scp ballu.csr alok@10.0.0.100:
scp ballu.key alok@10.0.0.100:
```

(Note: only the `.csr` — the public request — strictly needs to go to the admin; the `.key` stays private to the user in a real-world flow. Both were copied here for the lab.)

Step 3 — admin signs the CSR using the cluster's own CA

```bash
openssl x509 -req -in ballu.csr \
  -CA /etc/kubernetes/pki/ca.crt -CAkey /etc/kubernetes/pki/ca.key \
  -CAcreateserial -out ballu.crt -days 365
```

This produces `ballu.crt` — signed by the **same CA the cluster itself trusts**, which is exactly why Kubernetes will accept this certificate as a valid identity later.

```bash
root@manager:~# ll ballu.*
-rw-r--r-- 1 root root  981 ballu.crt
-rw-r--r-- 1 root root  875 ballu.csr
-rw------- 1 root root 1651 ballu.key
```

Step 4 — admin registers the user + binds them to a namespace context

```bash
kubectl config set-credentials ballu --client-certificate=ballu.crt --client-key=ballu.key
# User "ballu" set.

kubectl config set-context ballu-context --cluster=kubernetes --namespace=prod --user=ballu
```

`set-credentials` creates the **identity** (who ballu is, proven by cert+key). `set-context` binds that identity to a **specific cluster + namespace pairing** — this is what scopes ballu to `prod` specifically, not the whole cluster.

Writing the Role and RoleBinding YAML

Use `kubectl api-resources` to see what resources exist and can be referenced in a Role's rules.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: prod
  name: pod-creator
rules:
- apiGroups: [""]    # "" indicates the core API group
  resources: ["pods"]
  verbs: ["get", "watch", "list", "create"]
```

Later extended to cover more resource types in one Role:

```yaml
rules:
- apiGroups: ["", "apps/v1"]
  resources: ["pods", "deployments", "replicasets"]
  verbs: ["get", "watch", "list", "create"]
```

```bash
kubectl create -f prod-role.yaml
# role.rbac.authorization.k8s.io/pod-creator created

kubectl -n prod get roles
# NAME          CREATED AT
# pod-creator   2023-10-07T12:17:57Z
```

Now bind that Role to `ballu`:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: create-pods
  namespace: prod
subjects:
- kind: User
  name: ballu         # "name" is case sensitive
  namespace: prod
roleRef:
  kind: Role          # this must be Role or ClusterRole
  name: pod-creator    # must match the Role/ClusterRole you're binding to
  apiGroup: rbac.authorization.k8s.io
```

```bash
kubectl create -f role-bind.yaml
# rolebinding.rbac.authorization.k8s.io/create-pods created

kubectl -n prod describe rolebinding create-pods
# Role: pod-creator
# Subjects: User  ballu  prod
```

Handing the Finished Config Off to the User

> ⚠️ **Never hand out a config with admin creds still in it**
>
> The admin's `~/.kube/config` at this point still has BOTH the `kubernetes-admin` context and the new `ballu-context`. **Before handing this config file to ballu, strip out everything admin-related** — keep only `ballu-context`, set it as `current-context`, and keep only ballu's `client-certificate` / `client-key` entries under `users`.

```yaml
# What ballu's cleaned-up config should look like (relevant parts):
contexts:
- context:
    cluster: kubernetes
    namespace: prod
    user: ballu
  name: ballu-context
current-context: ballu-context   # must point at ballu, not admin
users:
- name: ballu
  user:
    client-certificate: /root/ballu.crt
    client-key: /root/ballu.key
```

Proving the Scoping Actually Works

Command	Result	Why
`kubectl get nodes`	`Forbidden: ... cannot list resource "nodes" ... at the cluster scope`	Nodes are cluster-scoped; user only has a namespaced Role
`kubectl get pods -n test` / `-n default`	`Forbidden: ... in the namespace "test"/"default"`	RoleBinding only covers `prod`
`kubectl get pods` (in prod, from context)	Works — lists pods	Role grants `get/list/watch` in `prod`
`kubectl delete pod ballupod` (in prod)	`Forbidden: ... cannot delete resource "pods" ...`	`delete` was never in the Role's verb list — least-privilege enforced verb-by-verb, not just namespace-by-namespace

✅ What I Practiced

- [ ] Walked through the full jump-host access model conceptually vs. direct master SSH.
- [ ] Generated a real cert-based user (`ballu`) end-to-end: key → CSR → admin-signed CRT.
- [ ] Wrote a `Role` scoped to `prod` with specific verbs, and a `RoleBinding` attaching it to `ballu`.
- [ ] Stripped admin credentials out of a handed-off kubeconfig and confirmed the user's `current-context` was the restricted one.
- [ ] Verified enforcement from the user's own machine: cluster-scope (nodes) forbidden, other namespaces forbidden, and even an in-scope-namespace verb not granted (delete) forbidden.

Merged into the master reference notes: k8s-master-notes.md §9 (Security & RBAC) — "Accessing a Cluster in Production & Certificate-Based User Onboarding" addendum.